Quantum computing

Lecture 1

April 15, 2026

Prof. Istvan Heckenberger Dr. Sebastian Halbig

heckenberger@mathematik.uni-marburg.de
Sebastian.Halbig@uni-marburg.de

Philipps Universität Marburg • Hans-Meerwein-Straße 6 • 35043 Marburg • Germany

First things first: organisation

- 2 lectures per week: 90 minutes each
  - Wed. 10.15–11.45 lecture hall 04A30 HS IV
  - Fri. 10.15–11.45 lecture hall 04A30 HS IV
- Getting in touch and getting help:
  - talk to us, feel free to ask all questions.
  - in-person meetings (offices are 04C34 and 04C35).
  - reach out by mail; return time is usually on the same day. Please write again if you do not get a timely answer.
  - use the ILIAS-forum.
- If things do not go according to plan (health issues, family issues, ...): reach out to us quickly so we can find a solution!
- We plan an excursion to the “Baby Diamond” quantum computer in Frankfurt during the semester.

Prerequisites

- You should know basic linear algebra (matrix-vector, matrix-matrix-multiplication, ...)
- The learning curve will be steep! The course requires at least 6-10h of work per week outside the classroom!

How to successfully pass the course

- Participate!
  - Come to the lectures and be active. “Active” could mean:
    - try to ask a question once per week
    - talk to us after/before the lecture
    - go to office hours
- Exercises
  - Each chapter of the lecture notes has exercises + study goals.
  - Training weekly with the exercises is *central* to your success!
  - There will not be any model solutions but feel free to discuss your ideas and problems with us.
  - Form study groups!

How to successfully pass the course

- Coursework
  - At least 3 discussions of 10-20 mins each about a block of exercises + questions from us to get an understanding of your knowledge.

Sample exercise block

Let $z_1 = 3 + 4i$ and $z_2 = 1 - i$.

1. Express $z_1 + z_2$, $z_1 z_2$, and z_1/z_2 in Cartesian form (i.e. as linear combinations $a + ib \in \mathbb{C}$).
2. Write z_2 in polar form $re^{2\pi i\alpha}$.
3. Find a complex number u such that $u^3 = z_2$.
4. Determine *integers* $p, q \in \mathbb{Z}$ such that $z_1^2 + pz_1 + q = 0$.
5. Prove that for any $z \in \mathbb{C} \setminus \{0\}$, we have $z^{-1} = \frac{\overline{z}}{\|z\|}$.

How to successfully pass the course

- Coursework
  - At least 3 discussions of 10-20 mins each about a block of exercises + questions from us to get an understanding of your knowledge.
  - Your task: prepare and understand(!) solutions to these exercises, upload them, and then present them.
  - The answers to our questions are part of your assessment!
  - You need to pass 2 of these discussions.
  - **Preparation**: 3 slots of weekly group meetings of 45 minutes for up to 8 persons tutored by Prof. Heckenberger and me. The time slots can also be book using ILIAS.
  - Place of the discussions: will be announced on ILIAS.
  - Slots for the discussions must be booked using ILIAS.

How to successfully pass the course

- Mock exams:
  - We will try to organise 2 mock exams during the semester.
  - **Note:** Inevitably the real exam will have additional topics (because there are lectures between the mock and the real exam).
- Exam:
  - Two time slots: July and September
  - Details will be communicated in due time (once we know how many students we have, ...).

Why should I study quantum computing?

- Quantum technology is an emerging branch, see e.g. `https://worldquantumday.org/`
- Quantum computers have many fascinating facets (engineering, physics, computer science, mathematics, ...),
- Topics for ongoing research.

  In our group: topological quantum computing.

What is the content of the course?

You might have heard: quantum computers will be able to break RSA encryption.

- First half: We will learn, from a math perspective, what this precisely means:
  - "languages" for quantum computing
    - Hilbert spaces, ...
    - ZX-calculus
  - quantum algorithms
    - Deutsch–Jozsa (see the lecture notes for a non-mathematical introduction)
    - Grover (for e.g. efficiently searching through data)
    - Shor (this is used to break encryption)

What is the content of the course?

Michaela Eichinger, PhD • Follower:in
Product Solutions Physicist @ Quantum Machines | I talk about quantum computing.
Zur Website
10 Std. •

Happy World Quantum Day! Everyone talks about logical qubits like it's a single metric. It's not.

When we look at what's actually needed to get to useful fault-tolerant quantum computing, there are multiple axes that all need to move forward at the same time.

First, quality. We need to keep pushing error rates down. Today we're in the 10^{-3} range for physical qubits. For the applications we care about, like quantum chemistry or materials simulation, we need logical error rates below 10^{-10} or even 10^{-12}. That's seven to nine orders of magnitude. No single technique gets us there. It takes better hardware, smarter error mitigation, and quantum error correction all working together.

Second, number. Hundreds to thousands of logical qubits. Which could still mean hundred thousands to millions of physical ones. It's not a qubit problem anymore. It's a systems engineering problem.

Third, and this one does not get as much attention: capabilities. A logical qubit that can only store information is a quantum memory. That's useful, but it's not computing. Actual computation needs state preparation, Clifford gates, non-Clifford gates, and real-time decoding with classical feedback that's fast enough so your computation doesn't die while waiting for the decoder.

A single number of "logical qubits" without context on quality, capabilities, and scale doesn't tell us much. So next time someone quotes a logical qubit number, better ask where they are on the map.

What is the content of the course?

- Second half
  - Universal quantum computing
    - Efficient Classical Simulation?
    - Universality
    - Equivalence of universal models
  - quantum error correction
    - Why do classical error-correction strategies fail
    - Threshold for error correction
    - 9-qubit code
    - toric code